

REVELA Pin Snapping: Revised Plan

Prepared October 8, 2026 · Based on a review of the REVELA backend (registry service, flags/detection service, places resolver, routes), MapPage.jsx and the database schema

1. Goal and success definition

Goal: every business pin is either on the correct business or clearly labeled as uncertain. No pin may look confident while being wrong.

Google Maps cannot list every registered business (many sari-sari stores and home businesses are not on it), so 100% exact placement is not achievable by any matching logic. The test should score confidence honesty, not only distance.

Observed result	Verdict
Pin within 30 m of the correct business	Pass
Pin 30 to 100 m off, labeled approximate or in review	Pass
Pin 30 to 100 m off, shown as confident	Fail
Pin on a different business in the same compound, shown as confident	Fail
Same compound, flagged for review	Pass
Wrong barangay, shown as confident	Fail
No pin, with a reason in the review queue	Pass
No pin, silently missing	Fail

Rule: confident-but-wrong pins on the test set must be zero. Agree the target percentage for exact matches with your tester after a baseline run.

2. What is wrong today

#	Finding	Where
1	Detection overwrites registry coordinates without checking matchStatus, coordSource or placeID, so approved pins are lost and rejected pins return. Coordinates it writes get no coordFetchedAt, so refresh and purge never see them.	flags/service.py, _match_registry_to_google
2	Manual drag-and-drop updates every registry row with the same businessName, sets no lock, and is overwritten by the next detection run.	update_flag_location
3	The name cleaner deletes category words and a substring shortcut returns 0.95. "Silva's Pharmacy" becomes <code>silva s</code> , which is a substring of <code>silva s apartment</code> . The possessive also leaves a stray <code>s</code> token.	_normalize_business_name, _name_similarity
4	The four-branch matcher accepts proximity (25 m) with a weak name, and a name score of 0.65 matches at any distance, which moves good pins.	_match_poi_to_registry
5	Matching is greedy per POI, not one-to-one. Duplicate cleanup deletes logs by name, which can remove a legitimate second branch.	_match_registry_to_google
6	Detection snaps are permanent: once a flag is Green, Orange, Black or Purple it is skipped forever, and detection matches never go to review.	run_detection
7	Sync reads only latitude and longitude of the existing row, so it cannot see locks or rejections. Existing coordinates always win, so corrected CSV coordinates are ignored. A rejected pin is re-resolved to the same wrong place.	sync_registry

#	Finding	Where
8	Import resolves only rows that have an address. The quota pre-check counts geocode quota, not Text Search budget.	upload_registry, sync_registry
9	Registry and pins are linked only by barangay plus name, in at least seven places. delete_business deletes every log with the same name and their inspection reports. update_business renames every matching log.	registry/service.py, flags/service.py, places_resolver.py
10	Resolver location bias and bounding box are unset by default. The refresh cap (3,000 per month) supports about 2,000 records, after which coordinates older than 28 days are purged and pins disappear.	places_resolver.py
11	The map receives no matchStatus, so uncertain pins look confident. Jitter shifts stacked pins by about 11 m times the square root of n.	MapPage.jsx
12	One API key appears to serve both browser and server, and the backend sends a spoofed Referer header.	_fetch_point_results
13	The legacy Nearby Search returns at most 60 results per grid point, so dense areas are truncated.	_fetch_point_results

3. Name matching redesign

Principle: keep the category as a separate signal and use it as a veto, instead of deleting it. Replace both existing matchers (the one in flags/service.py and `similarity()` in places_resolver.py) with one shared module, for example `api/utils/name_match.py`.

Category sources, both free:

- Registry side: the BPLO `lineOfBusiness` column.

- Google side: `primaryType` and `types`. These are Text Search Pro fields, the same tier as display name and location, so adding them to the field mask costs nothing extra. Do not add Enterprise fields such as rating, phone or website.

```
import re, difflib

LEGAL = {'inc', 'corp', 'corporation', 'co', 'company', 'ltd', 'opc', 'enterprise',
        'enterprises', 'trading', 'the', 'and', 'of'}

# Order matters: specific phrases first, generic ones (store) last.
CATEGORY_PATTERNS = [
    (r'\b(pharmacy|drug ?store|botika)\b', 'pharmacy'),
    (r'\b(apartments?|apartelle|boarding ?
house|dormitory|dorm|lodging|pension|inn|resort)\b', 'lodging'),
    (r'\b(bakery|bake ?shop|panaderi[ay])\b', 'bakery'),
    (r'\b(eatery|carinderi[ay]|karinderya|restaurant|grill|canteen|food ?
house|lomi(han)?|gotohan|cafe|coffee ?shop)\b', 'food'),
    (r'\b(salon|barber ?shop|beauty|spa)\b', 'personal_care'),
    (r'\b(hardware|construction supply)\b', 'hardware'),
    (r'\b(auto ?supply|motor ?parts|vulcanizing|talyer)\b', 'auto'),
    (r'\b(water refilling( station)?|water station|drinking water)\b',
'water'),
    (r'\b(clinic|dental|hospital|medical|laborator(y|ies))\b', 'health'),
    (r'\b(sari[- ]?sari|tindahan|grocery|mini ?mart|mart|supermarket|general
merchandise|store)\b', 'retail'),
]
TYPE_GROUPS =
{'pharmacy': 'pharmacy', 'drugstore': 'pharmacy', 'lodging': 'lodging',

'bakery': 'bakery', 'restaurant': 'food', 'cafe': 'food', 'hair_care': 'personal_car
e',

'beauty_salon': 'personal_care', 'hardware_store': 'hardware', 'car_repair': 'auto
',

'convenience_store': 'retail', 'grocery_store': 'retail', 'supermarket': 'retail'}

def parse_name(raw):
    s = (raw or '').lower().replace('&', ' and ')
    s = re.sub(r'[\u2019\x27]s\b', '', s)          # silva's -> silva
    s = re.sub(r'^a-z0-9 ]+', ' ', s)
    groups = set()
    for rx, g in CATEGORY_PATTERNS:
        if re.search(rx, s):
            groups.add(g); s = re.sub(rx, ' ', s)
    return [t for t in s.split() if t not in LEGAL], groups
```

```

def _overlap(ta, tb):
    if not ta or not tb: return 0.0
    small, big = (ta, tb) if len(ta) <= len(tb) else (tb, ta)
    hit = sum(1 for t in small
               if any(difflib.SequenceMatcher(None, t, u).ratio() >= 0.85 for
                      u in big))
    return hit / len(small)

def name_match(reg_name, poi_name, reg_line='', poi_types=()):
    ta, ga = parse_name(reg_name); tb, gb = parse_name(poi_name)
    ga |= parse_name(reg_line)[1] # BPLO
    lineOfBusiness
    gb |= {TYPE_GROUPS[t] for t in poi_types if t in TYPE_GROUPS} # Google
    types
    base = _overlap(ta, tb)
    if base == 0: return 0.0, 'no_shared_name'
    if min(len(ta), len(tb)) == 1: base = min(base, 0.70) # lone
    surname = weak
    if ga and gb:
        if ga & gb: return min(1.0, base + 0.10), 'ok'
        soft = 'retail' in (ga | gb)
        return min(base, 0.60 if soft else 0.45), 'category_conflict'
    return (base if not (ga or gb) else base * 0.90), 'ok'

```

The category list is a starting point. Tune it against real BPLO names and add the Taglish terms staff actually see.

Regression tests to write first:

Registry name	Candidate	Expected
Silva's Pharmacy	Silva's Apartment	No match
Jose Store	Jose Bakery	No match
Ana's	Banana Store	No match
Silva's Pharmacy	Silva Drugstore	Match
Mercedes Hardware	Mercedes Hardware & Construction Supply	Match
Silva's Pharmacy	Silva Store	Review

Known gap: initials (for example “J&M Trading” vs “JM Enterprises”) will not match and fall to the review queue, which is the right place for them.

4. Decision rules

Replace the four if-branches with these tiers:

Name score	Location evidence	Result
0.80 or higher	Same barangay, or within 300 m	Auto-snap
0.80 or higher	Different barangay or far away	Review
0.55 to 0.80	Any	Review
Below 0.55	Even if within 25 m	No match

Supporting rules:

- Proximity alone never produces a match.
- A far-away match never auto-snaps.
- Check barangay agreement using the existing `_get_barangay_id_by_coords`.
- Score all (POI, record) pairs, sort by score, and assign each side once, so two POIs cannot claim one record and the pin does not flip between runs.
- Exclude any place listed in `registry_rejected_places` for that business.
- Enforce one-to-one in code only for `placeIDKind = 'poi'`. Do not make `placeID` unique in the database, because address-type IDs can legitimately be shared.
- Locked rows (`matchStatus` approved, or `coordSource` manual) are never overwritten by detection, sync or refresh.

5. Database changes

The schema dump contains no data, so it is not a backup. Take a data backup before migrating.

Already present, no change needed: `placeID`, `placeIDKind`, `coordSource` (including `manual`), `coordFetchedAt`, `matchScore` and `matchStatus` on `official_registry`.

Stage A: additive and reversible

```
ALTER TABLE geospatial_logs
  ADD COLUMN businessID VARCHAR(50) COLLATE utf8mb4_unicode_ci NULL AFTER
  barangayID,
  ADD KEY idx_geo_business (businessID),
  ADD KEY idx_geo_place (placeID),
  ADD CONSTRAINT fk_geo_business FOREIGN KEY (businessID)
    REFERENCES official_registry (businessID)
```

```

ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE official_registry
  ADD KEY idx_reg_place (placeID),
  ADD COLUMN resolveKey CHAR(40) NULL;

CREATE TABLE registry_rejected_places (
  businessID VARCHAR(50) COLLATE utf8mb4_unicode_ci NOT NULL,
  placeID    VARCHAR(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  rejectedAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (businessID, placeID),
  CONSTRAINT fk_rej_business FOREIGN KEY (businessID)
    REFERENCES official_registry (businessID) ON DELETE CASCADE ON UPDATE
  CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

The new column type must match `official_registry.businessID` exactly (`varchar(50)`, `utf8mb4_unicode_ci`), or the foreign key fails. `resolveKey` stores a hash of name plus address so unchanged records are not looked up again.

Stage B: backfill only unambiguous matches

```

UPDATE geospatial_logs g
JOIN (SELECT barangayID, businessName, MIN(businessID) AS bid
      FROM official_registry
      GROUP BY barangayID, businessName
      HAVING COUNT(*) = 1) r
  ON r.barangayID = g.barangayID AND r.businessName = g.detectedName
SET g.businessID = r.bid
WHERE g.businessID IS NULL AND g.flagColor <> 'Red';

```

Then list what remains: duplicate names, businesses with more than one pin, and unlinked non-Red pins. Link these by hand.

Stage C: switch the code to write and read `businessID`, falling back to name only where the column is NULL.

Stage D: after a clean verification period, remove the name fallbacks. Consider a UNIQUE key on `geospatial_logs.businessID` only after the duplicate check is clean (some flows may keep more than one log per business).

Warning: `inspection_reports.targetID` holds a log ID with no foreign key. Any dedupe must re-point `targetID` to the surviving log instead of deleting logs.

Diagnostics to run first (these give the baseline for the test)

```

-- 1. How were coordinates obtained?
SELECT coordSource, matchStatus, COUNT(*) n FROM official_registry GROUP BY
1,2;
-- 2. Legacy rows with coordinates but no provenance
SELECT COUNT(*) FROM official_registry WHERE latitude IS NOT NULL AND
coordSource IS NULL;
-- 3. Pins stacked on one spot (centroid symptom)
SELECT ROUND(latitude,4) lat, ROUND(longitude,4) lng, COUNT(*) n FROM
official_registry
WHERE latitude IS NOT NULL GROUP BY 1,2 HAVING n > 1 ORDER BY n DESC LIMIT
20;
-- 4. Same name twice in a barangay
SELECT barangayID, businessName, COUNT(*) n FROM official_registry GROUP BY
1,2 HAVING n > 1;
-- 5. After Stage B: businesses with several pins, and unlinked non-Red pins
SELECT businessID, COUNT(*) n FROM geospatial_logs WHERE businessID IS NOT
NULL GROUP BY 1 HAVING n > 1;
SELECT logID, detectedName, barangayID, flagColor FROM geospatial_logs
WHERE businessID IS NULL AND flagColor <> 'Red';
-- 6. Pin vs registry coordinates that disagree (direct measure of drift)
SELECT r.businessID, r.businessName,
    ROUND(ST_Distance_Sphere(POINT(r.longitude, r.latitude), POINT(g.longitude,
g.latitude))) AS meters
FROM official_registry r JOIN geospatial_logs g ON g.businessID =
r.businessID
WHERE r.latitude IS NOT NULL AND g.latitude IS NOT NULL
HAVING meters > 5 ORDER BY meters DESC LIMIT 50;
-- 7. Coordinates the purge job would delete today
SELECT COUNT(*) FROM official_registry
WHERE coordSource IN ('places','geocode') AND coordFetchedAt < NOW() -
INTERVAL 28 DAY;

```

Legacy rows with a NULL `coordSource` should count as lowest priority so detection or a re-resolve can improve them. Do not bulk-label them as CSV, because that cannot be known.

6. Code changes by file

File	Changes
api/utils/name_match.py (new)	The shared matcher from section 3, with the regression tests.

File	Changes
flags/service.py	Remove <code>_normalize_business_name</code> , <code>_name_similarity</code> and the four-branch <code>_match_poi_to_registry</code> . Use scored, one-to-one matching with the section 4 tiers. <code>_match_registry_to_google</code> skips locked rows, stamps <code>coordSource</code> , <code>placeID</code> , <code>placeIDKind</code> , <code>coordFetchedAt</code> , <code>matchScore</code> and <code>matchStatus</code> , and dedupes by <code>businessID</code> , never by name. <code>update_flag_location</code> keys by <code>businessID</code> and sets <code>coordSource</code> manual and <code>matchStatus</code> approved. <code>_load_registry</code> joins by <code>businessID</code> . Matches below the auto tier go to review.
registry/service.py	Select <code>coordSource</code> and <code>matchStatus</code> in the sync query. Let a CSV coordinate overwrite unless the row is locked. Resolve rows when address or name exists. Pre-check the Text Search budget when the resolver is on. <code>delete_business</code> , <code>update_business</code> and <code>_sync_flag_color</code> work by <code>businessID</code> . On a name or address edit, clear <code>resolveKey</code> and send the row to re-resolve or review.
models/geospatial.py	<code>insert_green_flag</code> accepts and stores <code>businessID</code> (not reviewed yet).
places_resolver.py	Use <code>name_match</code> . Add <code>primaryType</code> and <code>types</code> to the field mask. Replace the unset bounding box with the municipality polygon, and set <code>PLACES_CENTER</code> . Downgrade results in a different barangay to review. Skip candidates in <code>registry_rejected_places</code> , and record the <code>placeID</code> on reject. Skip lookup when <code>resolveKey</code> is unchanged. <code>_update_pin</code> works by <code>businessID</code> .
flags/routes.py and <code>get_flags</code>	Return <code>matchStatus</code> and <code>coordSource</code> to the map.
MapPage.jsx	Style review and unresolved pins differently. Replace permanent jitter with click-to-spiderfy. Add a review-queue screen using the existing <code>/registry/review</code> endpoints (no UI for them was found in <code>MapPage.jsx</code>).

7. Cost plan

Prices below come from Google's published pricing as found on October 8, 2026. Confirm them in your own Cloud Console billing account before launch.

- Text Search Pro and Nearby Search Pro: 5,000 free calls per month, then \$32 per 1,000. The legacy Nearby and Text Search SKUs show the same figures.
- Geocoding: 10,000 free calls per month, then \$5 per 1,000.
- Free usage is per SKU, not pooled, and resets monthly.
- One dollar buys about 31 calls beyond the free tier.
- The price of Place Details at the Essentials tier (used by the refresh job with id and location only) was not verified. Check the pricing page row.

If you exceed the free quota, Text Search costs:

Calls in a month	Billable	Cost
5,000 or fewer	0	\$0
8,000 (large first import)	3,000	\$96
15,000 (heavy)	10,000	\$320

A big import is a one-time cost, since later months only pay for new or changed records.

My estimate for current usage: a full detection scan is under about 500 Nearby calls (up to about 160 grid points, up to 3 result pages each). With 2 scans a month that is under about 1,000, well inside the free 5,000. Resolver Text Search is one call per record without coordinates, capped at 2,500 per month by default.

Budget ceiling formula: set `TS_MONTHLY_CAP = 5,000 + budget_USD x 31.25`. A \$20 monthly ceiling is about 5,625 calls.

Refresh sizing: each record with Google-derived coordinates needs about 1.5 Place Details calls per month (refresh every 20 days). The default `PD_MONTHLY_CAP` of 3,000 supports about 2,000 records. Size it to your registry, or pins disappear after the 28-day purge. Whether you may keep Google-derived coordinates longer than that is a terms question to confirm with Google or an advisor before changing the design.

At your size: with 859 registered businesses and detection returning up to about 1,500 places, refreshing only the registry needs about 1,290 Place Details calls a month (859 x 1.5). Refreshing every Google-derived coordinate, including detected places, needs up to about 3,540 (2,359 x 1.5), so raise `PD_MONTHLY_CAP` from 3,000 to about 4,000. Place Details with only id and location is in the Essentials tier, whose free allowance is larger; the paid price was not verified. Expected spend at this scale is \$0 on the figures above.

Permanent coordinates: Google-derived coordinates stay temporary (refresh at 20 days, purge at 28). A coordinate confirmed on site (inspection_reports already stores verifiedLatitude, verifiedLongitude and verifiedAccuracy) or placed by an admin becomes coordSource manual, which is never refreshed or purged. Store the placeID permanently. Confirm with Google or an advisor that this fits their terms for your case.

Ways to spend less

- Resolve a record again only when its name or address changes (resolveKey). This also ends the reject-and-retry loop.
- Skip rows with no distinctive name words and send them straight to manual review.
- Dedupe identical name and address rows within one import.
- Run the 100-row test set once and keep the results instead of repeating calls.
- Keep the Pro-only field mask. You are billed at the highest SKU among requested fields.

Guardrails, in layers

1. App-level caps at or below the free thresholds, with a 10% buffer.
2. Per-API quotas in Google Cloud Console (a hard stop), plus budget alerts. Alerts alone do not stop spending.
3. A separate server key restricted by IP, and a browser key restricted by referrer.
4. Extend get_places_usage_today to show month-to-date calls and estimated overage in dollars.

Google also sells subscription plans (\$100, \$275 and \$1,200 per month). Consider them only if you regularly pay more than that on pay-as-you-go.

8. Rollout and gates

Phase	Work	Priority	Gate to continue
0	Data backup. Key separation, Cloud quotas and caps. Run diagnostics. Build the ground-truth set (about 100 records, including confusable pairs such as Silva's Pharmacy and Silva's Apartment).	Must-have	Caps verified in the Google console; baseline numbers recorded

Phase	Work	Priority	Gate to continue
1	Matcher, no schema change. Write the regression tests first and confirm they fail on the current matcher. Then add name_match with one-to-one assignment (score all pairs, assign each side once) and swap it into detection, reconcile and the resolver. Add primaryType and types to the field mask.	Must-have	Tests seen failing before the patch, then all table pairs pass; no POI or record is claimed twice
2	Locks and provenance, code only. Detection skips approved and manual rows and stamps provenance. Sync reads coordSource and matchStatus, lets CSV corrections win unless locked, and does not re-resolve rows with matchStatus rejected. Fix the import address gate and the Text Search quota pre-check. Detection skips any POI whose placeID is already held by a registry row of kind poi, so registered businesses are not red-flagged. Interim guard in delete_business and update_business: do not delete or rename logs shared with another registry row of the same name in that barangay.	Must-have	A re-import moves no locked or approved pin; a rejected pin does not return; deleting one business removes no other pin

Phase	Work	Priority	Gate to continue
3	Schema. Stages A to C of the migration (businessID link, rejected-places table, resolveKey), then switch the name-based joins to businessID and record rejected placeIDs.	Must-have, after phases 1 and 2	Unlinked-pin and duplicate-pin reports reviewed; joins read businessID first
4	Barangay check, review screen, map styling for review and unresolved pins, resolver config (polygon, center), and refresh cap sized to the registry.	Can wait for the matcher fix; required before the acceptance run	Review queue works end to end; refresh cap covers the registry
5	Full acceptance run against the section 1 table. Then Stage D cleanup.	Must-have	Zero confident-wrong pins on the test set

No-go items

- Do not run the Stage A migration before a data backup exists.
- Do not turn on the new auto-snap tiers before the regression tests and one-to-one assignment are in place.
- Do not let detection, sync or refresh write coordinates to a row with matchStatus approved or coordSource manual.
- Do not add Enterprise-tier fields (rating, phone, website, hours, reviews) to Text Search. It raises the SKU and cuts the free allowance from 5,000 to 1,000 calls.
- Do not delete geospatial logs in any dedupe without re-pointing inspection_reports.targetID to the surviving log.
- Do not make placeID unique in the database. Address-type IDs can legitimately be shared.
- Do not bulk-label legacy rows with a NULL coordSource as CSV.
- Do not raise API caps above the dollar ceiling you agreed.
- Do not remove the name-based fallbacks (Stage D) until the unlinked-pin report is empty.
- Do not run the acceptance test with a different matcher or configuration than the one going live.

9. Acceptance test procedure

1. Choose about 100 registry records across barangays. Include confusable pairs, same-compound businesses, and some that are not on Google.
2. Verify each true location by hand (Google Maps or a site check) and save as a CSV.
3. A script imports the set, runs detection, and reports exact, area, unresolved and wrong counts against the section 1 table.
4. Re-run after every phase. Keep review-queue approvals and rejections as additional labeled examples.
5. Give the tester the verdict table before the run.

10. Assumptions and open questions

Not reviewed: the registry data itself (the dump has no rows), `insert_green_flag` in `models/geospatial.py`, `get_flags`, and the rest of `MapPage.jsx`. Some details above may need adjusting once those are checked.

Open questions that change the plan:

- Decided: the registry holds 859 businesses, so the one-time Text Search cost is about 859 calls, inside the free 5,000.
- Is the browser key also used by the backend?
- Decided: keep the 28-day refresh and purge for Google-derived coordinates, and make field-verified and admin-placed coordinates permanent (`coordSource` manual). Confirm with Google's terms or an advisor that this fits your case.
- Decided: one current pin per business. Run diagnostic 5 first, clean up any duplicates, then add the `UNIQUE` key in Stage D.